CYBERSECURITY

ASSESSMENT REPORT

KIIS Bank Portal

JWT Authentication Bypass & Privilege Escalation Research

Report Date: June 07, 2026

Classification: CONFIDENTIAL — Research & Educational Use Only

Authored by

Himamanth

Cybersecurity Professional | Penetration Tester | AppSec Researcher

github.com/Himamanth997/kiis-bank-portal

Target Application	KIIS Bank Portal (Full-Stack Banking Simulation)
Repository	github.com/Himamanth997/kiis-bank-portal
Technology Stack	Python FastAPI · React/Vite · PostgreSQL · Docker · JWT (HS256)
Assessment Type	Source Code Review · API Security Testing · Intentional Vulnerability Lab
Findings Summary	**2 Critical** **2 High** **2 Informational**

00 Table of Contents

01 Executive Summary

This report documents a comprehensive security assessment of the KIIS Bank Portal, a full-stack banking simulation application developed as a cybersecurity education and demonstration platform. The assessment covers source-code review, API endpoint analysis, JWT (JSON Web Token) security mechanisms, role-based access control (RBAC), and intentionally embedded vulnerabilities designed for security awareness training.

The portal was deliberately constructed with multiple real-world authentication flaws to enable hands-on demonstration of attack vectors using industry-standard tools such as Burp Suite. This approach reflects advanced understanding of offensive security methodology, the design of controlled lab environments, and the ability to communicate complex vulnerabilities to diverse audiences.

Severity	Count	Key Findings
CRITICAL	2	JWT alg:none bypass, Role escalation via token tampering
HIGH	2	Weak signing secret, Hardcoded credentials in source
MEDIUM	0	—
LOW / INFO	2	Permissive CORS, JWT role stored in payload

Key Takeaway: The KIIS Bank Portal successfully demonstrates attacker-perspective thinking by embedding real CVE-class vulnerabilities (CWE-347, CWE-522, CWE-266) within a fully functional banking application. This reflects hands-on expertise in JWT attack chains, RBAC bypass techniques, and security-by-design principles.

02 Project Overview & Architecture

The KIIS Bank Portal is a production-grade full-stack web application simulating a modern digital banking environment. It integrates a React/Vite frontend, a Python FastAPI backend, PostgreSQL persistence layer, and Docker-based deployment orchestration with Nginx reverse proxy.

Technology Stack

Layer	Technology	Purpose
Frontend	React 18 · Vite · TailwindCSS	Customer & Staff portal UI with biometric/OTP simulation
Backend API	Python FastAPI · Uvicorn	REST API with JWT auth, RBAC, and security lab endpoints
Auth	python-jose · bcrypt	JWT generation/validation · Password hashing
Database	PostgreSQL 15 · SQLAlchemy ORM	User accounts, transactions, vKYC, fixed deposits
DevOps	Docker · docker-compose · Nginx	Multi-container deployment, reverse proxy, TLS-ready
Cloud	Render.com · render.yaml	Zero-config PaaS deployment with environment variables

API Architecture & Role Hierarchy

The backend exposes a structured REST API with four permission tiers enforced via a custom **RoleChecker** dependency injected into each FastAPI route. Roles are stored in the PostgreSQL database and are also embedded in JWT payloads — the latter being an intentional vulnerability for lab demonstration purposes.

Role	Accessible Endpoints	Privilege Level
customer	/api/customer/* — Accounts, Transfers, Fixed Deposits, vKYC	Low
employee	/api/employee/* — Customer lookup, transaction view	Medium
manager	/api/manager/* — Reporting, account oversight	High
admin	/api/admin/* — User management, account provisioning	Full

03 Vulnerability Findings

The following vulnerabilities were identified through source code analysis and live API testing. Each finding includes a proof-of-concept reproduction path, CVSS assessment, CWE classification, and actionable remediation guidance.

VLN-01 | JWT Algorithm Confusion — alg:none Bypass

CRITICAL

CVSS: 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H) **CWE:** CWE-347

Description	The backend accepts JSON Web Tokens with the algorithm set to 'none', meaning the server processes unsigned tokens as if they were cryptographically valid. The **lab_jwt.py** module explicitly implements this bypass: when SECURITY_LAB_MODE is active, any JWT with alg=none is decoded and trusted without signature verification.
Impact	A remote unauthenticated attacker can forge a JWT claiming any username and any role (including 'admin'). By making a single GET request to the forge endpoint, the attacker obtains a token accepted by the admin API. This gives full access to user enumeration, account provisioning, and all privileged operations — complete authentication bypass.
Evidence / PoC	`GET /api/lab/forge-token/none?username=Jane%20Doe&role=admin → { "token": "eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOiJKYW5lIERvZSIsInJvbGUiOiJhZG1pbiIsImxhYl9ieXBhc3MiOnRydWV9." } GET /api/admin/users Authorization: Bearer <forged_token> → HTTP 200 OK [{ "id": 1, "username": "Jane Doe", ... }, ...]`
Remediation	Enforce a strict algorithm whitelist: **jwt.decode(token, secret, algorithms=['HS256'])**. Reject any token where the header algorithm does not match exactly. Never accept 'none' as a valid algorithm in any code path, including fallback or error-handling logic.

VLN-02 | JWT Role Claim Privilege Escalation

CRITICAL

CVSS: 9.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H) **CWE:** CWE-266

Description	The **_user_from_token_payload()** function in auth.py accepts an optional **trust_token_role=True** parameter. When this flag is set, the user's role is overridden by the 'role' field in the JWT payload rather than being fetched from the database. This means an attacker who can sign or forge a token can escalate to any role simply by modifying the payload claim.
Impact	A low-privileged user (customer) who obtains or forges a JWT with role='admin' gains full administrative access. Since the role is taken from the token without database validation, the privilege escalation requires no server-side changes — only a crafted token. This violates the principle of least privilege and centralized authorization.
Evidence / PoC	`# Auth.py — vulnerable code path: if trust_token_role and payload.get('role'): user.role = payload.get('role') # ← attacker controlled! # Attacker payload: { "sub": "jane_doe", "role": "admin", "exp": 9999999999 }`
Remediation	Never trust security-sensitive claims from client-controlled tokens. Always fetch the user's current role from the database after token validation: **db.query(User).filter(...).first().role**. Remove the trust_token_role parameter entirely from production code.

VLN-03 | Weak JWT Signing Secret — Brute-Force / Dictionary Attack Risk

HIGH

CVSS: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) **CWE:** CWE-522

Description	The configuration in **config.py** defines a fallback weak secret: **LAB_JWT_WEAK_SECRET = 'kiis-lab-weak-secret'**. The lab JWT bypass module also attempts to decode tokens using this weak secret as an alternative validation path. If an operator forgets to set a strong SECRET_KEY environment variable, the default fallback ('09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7') may also be used.
Impact	An attacker who captures a JWT (e.g., via network interception or log exposure) can use offline brute-force tools such as **hashcat** or **jwt_tool** to crack the weak secret and forge arbitrary tokens. Successful cracking grants token-signing capability equivalent to the server itself.
Evidence / PoC	`# Offline attack with hashcat: hashcat -a 0 -m 16500 <jwt_token> wordlist.txt # Or with jwt_tool: python3 jwt_tool.py <token> -C -d wordlist.txt # Weak secret found in config.py: LAB_JWT_WEAK_SECRET = 'kiis-lab-weak-secret'`
Remediation	Generate cryptographically secure secrets using **secrets.token_hex(64)** (minimum 256-bit). Store secrets exclusively in environment variables or secrets managers (AWS Secrets Manager, HashiCorp Vault). Implement automatic key rotation and short token expiry (15–60 minutes).

VLN-04 | Hardcoded Fallback Secret Key in Source Code	HIGH
CVSS: 7.2 (AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H)	**CWE:** CWE-321
Description	The **config.py** file contains a hardcoded fallback JWT secret: **'09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7'**. This secret is committed to the public GitHub repository, meaning anyone who reads the source code can use it to sign arbitrary JWTs — even without access to the running server.
Impact	Anyone with access to the GitHub repository (which is public) can immediately derive the server's JWT signing secret if the production environment variable is not set. This completely undermines the confidentiality of the authentication system. Even if rotated, git history preserves the exposure permanently.
Evidence / PoC	`# config.py — line visible in public repo: SECRET_KEY = os.getenv('SECRET_KEY') or os.getenv( 'JWT_SECRET', '09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7' # ← hardcoded! ) # Anyone can forge tokens with this key immediately.`
Remediation	Never include secrets in source code — not even as fallback defaults. Use **os.getenv('SECRET_KEY')** and raise a startup error if the variable is absent. Add **.env** to **.gitignore**, use pre-commit hooks to scan for secrets (truffleHog, git-secrets), and rotate any exposed secrets immediately.

VLN-05 | Overly Permissive CORS Configuration

INFO

CVSS: 4.3 (AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N) **CWE:** CWE-942

Description	The CORS middleware in **main.py** uses a broad regex: **allow_origin_regex=r'https://.*\.(onrender\.com|trycloudflare\.com)'**. This allows any subdomain of onrender.com or trycloudflare.com — including attacker-controlled applications deployed on these shared platforms — to make credentialed cross-origin requests to the API.
Impact	A malicious actor can deploy a phishing page at any-name.onrender.com and make authenticated cross-origin API calls to the target. This bypasses Same-Origin Policy restrictions and could be combined with CSRF or social engineering to steal data from authenticated users.

Evidence / PoC	`# main.py:`
	`allow_origin_regex=r'https://.*\.(onrender\.com|trycloudflare\.com)' # → Permits: evil-attacker.onrender.com ← attacker-controlled!`
	`allow_credentials=True # ← Makes this especially dangerous`

Remediation	Restrict CORS to a whitelist of exact, known origins. In production, specify the precise deployment URL rather than a wildcard pattern. Remove **allow_credentials=True** unless strictly required, and never combine wildcard origin patterns with credentials.

VLN-06 | User Role Embedded in JWT Payload (Design Weakness)

INFO

CVSS: 3.1 (AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N) **CWE:** CWE-284

Description	The login endpoint encodes the user's **role** directly into the JWT payload: **data={'sub': user.username, 'role': user.role, 'email': user.email}**. While the legitimate code path re-validates the role from the database, the role claim in the token creates an attack surface that is exploited by VLN-02. It also means that role changes made after token issuance are not reflected until the token expires.
Impact	Combined with VLN-02, this design allows privilege escalation. Even without VLN-02, embedding role in the token creates a stale-authorization window: if a user is demoted, their existing token retains the elevated role until expiry. This violates the principle of revocability in access control.
Evidence / PoC	`# Login response JWT payload (decoded): { 'sub': 'jane_doe', 'role': 'customer', 'email': 'jane@kiis.bank', 'exp': ... } # If role='admin' is inserted: privilege escalation (see VLN-02)`
Remediation	Remove the role claim from the JWT entirely. During each request, look up the current role from the database using only the **sub** (username) claim. Implement token revocation via a Redis-backed blocklist for high-security role changes.

04 Exploitation Walkthrough — Burp Suite Demo

The project includes a complete Burp Suite workflow demonstrating the JWT bypass chain. This section documents the end-to-end attack path as it would be performed in a real penetration test engagement.

01	**Step 1 — Intercept Login Request** Configure Burp Proxy (127.0.0.1:8080) and install the CA certificate. Navigate to the portal and authenticate as a low-privilege customer (jane_doe / customer123). Intercept the POST /api/auth/login request in Burp HTTP History and extract the access_token from the JSON response.
02	**Step 2 — Verify Access Restriction** Send GET /api/admin/users with the legitimate customer token in the Authorization: Bearer header. Observe HTTP 403 Forbidden — confirming RBAC is enforced for normal tokens.
03	**Step 3 — Forge Unsigned Token (alg:none)** With SECURITY_LAB_MODE enabled, request GET /api/lab/forge-token/none?username=Jane%20Doe&role;=admin. The server returns an unsigned JWT with role='admin'. Copy this token.
04	**Step 4 — Privilege Escalation** Replace the Authorization header with Bearer and re-send GET /api/admin/users. Observe HTTP 200 OK with the complete user database — full privilege escalation achieved without credentials.
05	**Step 5 — Document & Report** Screenshot all HTTP exchanges in Burp Repeater. Annotate with CVE references (CVE-2015-9235 for alg:none class vulnerabilities). Package findings into this structured report for stakeholder presentation.

05 Secure Architecture Recommendations

The following hardening measures should be applied when transitioning this application from educational lab to production deployment.

JWT Hardening

→ Enforce algorithm whitelist: algorithms=['HS256'] with no fallback
→ Set token expiry to 15 minutes; use refresh token rotation
→ Generate SECRET_KEY with secrets.token_hex(64) — never hardcode
→ Implement token revocation via Redis blocklist for critical role changes
→ Remove role claim from payload; always resolve from database on each request

Authentication

→ Implement account lockout after 5 failed login attempts (429 + exponential backoff)
→ Add multi-factor authentication for admin and manager roles
→ Log all authentication events with IP, timestamp, and user agent
→ Hash passwords with bcrypt (cost factor >= 12) — already implemented, maintain it

Authorization

→ Never trust role from JWT payload; always query the database
→ Implement attribute-based access control (ABAC) for fine-grained permissions
→ Enforce resource ownership checks on all customer endpoints
→ Conduct regular access reviews for privileged accounts

Infrastructure & Secrets

→ Store all secrets in environment variables; never commit to version control
→ Use a secrets manager (AWS Secrets Manager, HashiCorp Vault) in production
→ Restrict CORS to exact deployment URLs; remove wildcard patterns
→ Enable HTTPS-only with HSTS headers; configure strict Content-Security-Policy
→ Run container images as non-root; use read-only filesystems where possible

06 OWASP Top 10 (2025) Mapping

All identified vulnerabilities map directly to the OWASP Top 10 (2025) — the latest industry-standard framework for web application security risks, released January 2026. Key 2025 changes include: Security Misconfiguration rising to #2, new categories for Software Supply Chain Failures (A03) and Mishandling of Exceptional Conditions (A10), and Cryptographic Failures shifting to #4.

Finding	OWASP 2025 Category	ID
JWT alg:none Bypass	A04:2025 — Cryptographic Failures	A04
JWT Role Escalation	A01:2025 — Broken Access Control	A01
Weak Signing Secret	A02:2025 — Security Misconfiguration	A02
Hardcoded Secret Key	A02:2025 — Security Misconfiguration	A02
Permissive CORS	A02:2025 — Security Misconfiguration	A02
Role in JWT Payload	A07:2025 — Authentication Failures	A07

This mapping reflects the OWASP Top 10 (2025) — the first update since 2021, released January 2026. Staying current with the latest OWASP edition demonstrates awareness of evolving threat landscapes, including newly recognised risks like Software Supply Chain Failures (A03) and Mishandling of Exceptional Conditions (A10), which are reshaping how security teams prioritise remediation efforts.

07 Conclusion & Researcher Profile

The KIIS Bank Portal demonstrates a sophisticated command of both offensive security research and the engineering skills required to build realistic, production-grade vulnerable-by-design systems. The project is not a simple toy — it implements Docker orchestration, multi-role RBAC, ACID-compliant database transactions, Nginx reverse proxy configuration, and a Vite/React frontend — all of which required substantial full-stack engineering knowledge.

From a security research perspective, the embedded vulnerabilities accurately replicate real CVE-class issues (alg:none bypass is catalogued under CVE-2015-9235). The Burp Suite integration guide, JWT forge endpoints, and lab mode toggle demonstrate hands-on penetration testing methodology. This combination of build and break skill is rare and highly valued in the cybersecurity industry.

Skill Domain	Evidence from Project
Penetration Testing	JWT alg:none exploit, privilege escalation PoC, Burp Suite workflow
Secure Code Review	Identified CWE-347/266/522/321/942 across Python FastAPI codebase
Application Security	OWASP Top 10 mapping, RBAC design analysis, CORS misconfiguration
Full-Stack Engineering	React · FastAPI · SQLAlchemy · PostgreSQL · Docker · Nginx
Security Architecture	Remediation design, threat modeling, least-privilege recommendations
Communication	Structured CVE-style reporting, educational lab design, stakeholder-ready output

This report was researched and authored by **Himamanth** — a cybersecurity professional with demonstrated expertise in JWT attack chains, API penetration testing, and secure application design. The KIIS Bank Portal project represents hands-on, production-quality security research that goes beyond CTF challenges into real-world engineering and threat modeling. Source code, deployment configuration, and full exploit documentation are available at github.com/Himamanth997/kiis-bank-portal.